Cours de Python Avancé — Licence 3 Intelligence Artificielle (30h)

Oct, 2026 · @Elias

Enseignant : Elias BADJI · **Établissement :** ISM École d'ingénieurs

Présentation générale

Ce module de 30h fait découvrir aux étudiants les bibliothèques Python incontournables, en trois blocs : l'analyse de données, l'intelligence artificielle et le développement de jeux. Il compte 7 séances de 4h et une séance de soutenance de 2h. Chaque séance suit le même rythme : 1h15 de découverte guidée, 15 min de pause, 2h de TP et 30 min de démonstration des résultats.

Objectifs pédagogiques. À la fin du module, l'étudiant sait :

- installer et gérer un environnement Python avec ses bibliothèques ;
- charger, nettoyer, analyser et visualiser un jeu de données réel ;
- entraîner et évaluer un modèle de Machine Learning puis de Deep Learning ;
- utiliser des modèles d'IA pré-entraînés pour le texte et l'image ;
- programmer un jeu 2D et y intégrer un agent intelligent ;
- lire la documentation d'une bibliothèque inconnue et s'en servir en autonomie.

Prérequis. Bases de Python (variables, boucles, fonctions, listes, dictionnaires, classes simples).

Environnement. Python 3.12+, `uv` ou Anaconda, Jupyter Notebook ou VS Code, Git. Google Colab sert de solution de secours pour les postes sans GPU ou sans droits d'installation.

Séance	Durée	Bloc	Bibliothèques découvertes	TP
1	4h	Données	Jupyter, NumPy	Statistiques sur données météo
2	4h	Données	Pandas	Analyse d'un jeu de données réel
3	4h	Données	Matplotlib, Seaborn, Plotly, Streamlit	Tableau de bord interactif
4	4h	IA	scikit-learn	Prédiction et classification
5	4h	IA	PyTorch, torchvision, OpenCV	Reconnaissance d'images
6	4h	IA	spaCy, Hugging Face	Analyse de sentiments,

Séance	Durée	Bloc	Bibliothèques découvertes	TP
			Transformers	chatbot
7	4h	Jeux	Pygame, Gymnasium	Jeu 2D avec agent IA
8	2h	Projet	—	Soutenances

Séance 1 — L'écosystème Python et NumPy (4h)

Objectif : installer un environnement de travail complet et découvrir NumPy, la fondation de toutes les bibliothèques de données et d'IA.

Contenu
Panorama de l'écosystème : PyPI, environnements, Jupyter, Colab
Découverte de NumPy
Pause
TP1
Démonstrations et synthèse

Bibliothèques et outils.

Outil	À quoi il sert
uv / pip	Installer des bibliothèques dans un environnement isolé
Jupyter	Coder, exécuter et documenter dans un même carnet
NumPy	Tableaux multidimensionnels et calcul numérique rapide

Notions abordées. Création de tableaux (`array`, `arange`, `linspace`, `zeros`), forme et type (`shape`, `dtype`), indexation et masques booléens, opérations vectorisées, broadcasting, statistiques (`mean`, `std`, `argmax`), nombres aléatoires reproductibles, comparaison de vitesse avec une boucle Python.

```
import numpy as np

rng = np.random.default_rng(42)
temperatures = rng.normal(loc=29, scale=3, size=(12, 30))  # 12 mois x 30 jours

print(temperatures.shape)            # (12, 30)
moyennes = temperatures.mean(axis=1) # moyenne par mois
```

```
print(moyennes.argmax() + 1)         # mois le plus chaud
print((temperatures > 35).sum())     # nombre de jours de canicule
```

TP1 — Statistiques sur des données météo. À partir d'un tableau de températures simulées ou réelles pour Dakar, calculer les moyennes mensuelles, détecter les jours extrêmes, normaliser les données et comparer le temps d'exécution entre une boucle Python et NumPy.

Séance 2 — Analyse de données avec Pandas (4h)

Objectif : passer d'un fichier brut à des réponses chiffrées, en maîtrisant le cycle charger, nettoyer, transformer, agréger.

Contenu
Découverte de Pandas
Pause
TP2
Démonstrations et synthèse

Bibliothèques.

Bibliothèque	À quoi elle sert
Pandas	Tableaux de données (DataFrame) : lecture, nettoyage, agrégation
openpyxl	Lire et écrire des fichiers Excel avec Pandas
Polars (ouverture)	Alternative plus rapide pour les gros volumes

Notions abordées. `Series` et `DataFrame`, lecture CSV/Excel/JSON, exploration (`head`, `info`, `describe`), sélection avec `loc` et `iloc`, filtres, valeurs manquantes (`isna`, `fillna`, `dropna`), doublons, colonnes calculées, dates, `groupby` et `agg`, `merge`, `pivot_table`, export.

```
import pandas as pd

df = pd.read_csv("ventes.csv", parse_dates=["date"])
df = df.dropna(subset=["montant"]).drop_duplicates()
df["mois"] = df["date"].dt.month

resume = (df.groupby(["region", "mois"])["montant"]
            .agg(["sum", "mean", "count"])
            .sort_values("sum", ascending=False))
print(resume.head(10))
```

TP2 — Analyse d'un jeu de données réel. Chaque binôme choisit un jeu de données ouvert (Titanic, données de l'ANSD sur le Sénégal, prix des denrées, trafic d'un site). Il le nettoie, répond à cinq questions métier posées par l'enseignant et rédige ses conclusions dans le notebook.

Séance 3 — Visualisation et tableaux de bord (4h)

Objectif : choisir le bon graphique pour chaque question et transformer une analyse en application web interactive, sans écrire de HTML.

Contenu
Matplotlib et Seaborn : graphiques statiques
Plotly et Streamlit : interactivité
Pause
TP3
Démonstrations des tableaux de bord

Bibliothèques.

Bibliothèque	À quoi elle sert
Matplotlib	Graphiques de base, contrôle fin de chaque élément
Seaborn	Graphiques statistiques en une ligne (distributions, corrélations)
Plotly Express	Graphiques interactifs (zoom, survol, cartes)
Streamlit	Application web de données en pur Python

Notions abordées. Figure et axes, courbes, barres, histogrammes, nuages de points, boîtes à moustaches, carte de corrélation, sous-graphiques, sauvegarde en image, graphiques interactifs, widgets Streamlit (`selectbox`, `slider`), mise en cache, déploiement gratuit sur Streamlit Community Cloud.

```
import streamlit as st
import pandas as pd
import plotly.express as px

df = pd.read_csv("ventes.csv", parse_dates=["date"])
region = st.selectbox("Région", sorted(df["region"].unique()))
filtre = df[df["region"] == region]

st.metric("Chiffre d'affaires", f"{filtre['montant'].sum():,.0f} FCFA")
st.plotly_chart(px.line(filtre, x="date", y="montant"))
```

Lancement : `streamlit run app.py`.

TP3 — Tableau de bord interactif. Reprendre le jeu de données du TP2 et produire d'abord trois graphiques Seaborn commentés, puis une application Streamlit avec au moins deux filtres, trois indicateurs et deux graphiques Plotly.

Séance 4 — Machine Learning avec scikit-learn (4h)

Objectif : entraîner, évaluer et comparer des modèles de Machine Learning classiques avec une API unique, `fit` puis `predict`.

Contenu
Découverte de scikit-learn
Pause
TP4
Comparaison des résultats entre binômes

Bibliothèques.

Bibliothèque	À quoi elle sert
scikit-learn	Prétraitement, modèles, validation et métriques
joblib	Sauvegarder et recharger un modèle entraîné
XGBoost (ouverture)	Gradient boosting performant sur données tabulaires

Notions abordées. Apprentissage supervisé et non supervisé, découpage entraînement/test, normalisation et encodage (`StandardScaler`, `OneHotEncoder`), `Pipeline`, régression linéaire, arbres et forêts aléatoires, k plus proches voisins, k-means, validation croisée, `GridSearchCV`, métriques (exactitude, précision, rappel, F1, matrice de confusion, RMSE), surapprentissage.

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

modele = make_pipeline(StandardScaler(), RandomForestClassifier(n_estimators=200))
modele.fit(X_train, y_train)
print(classification_report(y_test, modele.predict(X_test)))
```

TP4 — Prédiction et classification. Partie 1 : prédire la survie des passagers du Titanic en comparant au moins trois modèles. Partie 2 : segmenter des clients avec k-means et interpréter les groupes obtenus. Bonus : intégrer le meilleur modèle dans l'application Streamlit du TP3.

Séance 5 — Deep Learning et vision par ordinateur (4h)

Objectif : construire et entraîner un réseau de neurones, puis réutiliser un modèle pré-entraîné pour reconnaître des images.

Contenu
PyTorch : tenseurs, réseau, boucle d'entraînement
OpenCV et modèles pré-entraînés
Pause
TP5 (sur Colab avec GPU si besoin)
Démonstrations

Bibliothèques.

Bibliothèque	À quoi elle sert
PyTorch	Tenseurs, calcul de gradients, réseaux de neurones
torchvision	Jeux d'images (MNIST, CIFAR-10) et modèles pré-entraînés (ResNet)
OpenCV	Lire, transformer et analyser des images et des flux webcam
Pillow	Manipulation simple d'images
Ultralytics YOLO (ouverture)	Détection d'objets en temps réel

Notions abordées. Tenseurs et GPU, `nn.Module`, couches denses et convolutives, fonction de perte, optimiseur, boucle d'entraînement, `Dataset` et `DataLoader`, courbes d'apprentissage, transfer learning, prétraitement d'images avec OpenCV (niveaux de gris, redimensionnement, contours, détection de visages).

```
import torch
from torch import nn

modele = nn.Sequential(
    nn.Flatten(),
    nn.Linear(28 * 28, 128), nn.ReLU(),
    nn.Linear(128, 10),
)
optimiseur = torch.optim.Adam(modele.parameters(), lr=1e-3)
perte = nn.CrossEntropyLoss()
```

```
for images, etiquettes in train_loader:
    optimiseur.zero_grad()
    loss = perte(modele(images), etiquettes)
    loss.backward()
    optimiseur.step()
```

TP5 — Reconnaissance d'images. Partie 1 : entraîner un réseau sur MNIST et dépasser 97 % d'exactitude. Partie 2 : réutiliser un ResNet pré-entraîné pour classer des photos prises par les étudiants. Bonus : détecter les visages en direct depuis la webcam avec OpenCV.

Séance 6 — Traitement du langage et IA générative (4h)

Objectif : analyser du texte avec des outils classiques, puis utiliser des modèles de langage pré-entraînés pour classer, résumer et dialoguer.

Contenu
spaCy : traitement linguistique
Hugging Face et API de modèles de langage
Pause
TP6
Démonstrations et discussion éthique

Bibliothèques.

Bibliothèque	À quoi elle sert
spaCy	Tokenisation, lemmatisation, entités nommées, en français et en anglais
Hugging Face Transformers	Milliers de modèles pré-entraînés via `pipeline`
Hugging Face Datasets	Jeux de données textuels prêts à l'emploi
SDK d'API LLM (Anthropic, OpenAI ou Mistral)	Appeler un grand modèle de langage depuis Python
Gradio	Interface web de démonstration en quelques lignes

Notions abordées. Tokenisation, lemmes, stopwords, entités nommées, représentation vectorielle (TF-IDF, embeddings), modèles Transformer, analyse de sentiments, résumé, traduction, ingénierie de prompts, notion de RAG, limites et biais des modèles, langues africaines (modèles multilingues, wolof).

```
from transformers import pipeline

sentiment = pipeline("sentiment-analysis",
                     model="nlptown/bert-base-multilingual-uncased-sentiment")
print(sentiment("Ce cours de Python est vraiment passionnant !"))
```

```
resume = pipeline("summarization")
print(resume(long_texte, max_length=60)[0]["summary_text"])
```

TP6 — Analyse de sentiments et chatbot. Partie 1 : extraire les entités nommées d'articles de presse avec spaCy. Partie 2 : classer des avis clients avec un modèle Hugging Face et mesurer son exactitude sur 100 avis annotés. Partie 3 : construire avec Gradio un petit assistant qui répond aux questions sur le règlement de l'école.

Séance 7 — Jeux avec Pygame et agents intelligents (4h)

Objectif : programmer un jeu 2D complet, puis faire jouer une IA à la place de l'humain, ce qui relie programmation événementielle et apprentissage par renforcement.

Contenu
Pygame : boucle de jeu, sprites, collisions
Gymnasium : environnement, agent, récompense
Pause
TP7
Tournoi de démonstration

Bibliothèques.

Bibliothèque	À quoi elle sert
Pygame	Fenêtre, affichage, clavier, sons et collisions pour jeux 2D
Gymnasium	Environnements standard pour entraîner des agents (CartPole, FrozenLake)
Stable-Baselines3	Algorithmes de renforcement prêts à l'emploi (PPO, DQN)
Arcade (ouverture)	Alternative moderne à Pygame

Notions abordées. Boucle de jeu (événements, mise à jour, affichage), images par seconde, classes `Sprite` et groupes, collisions, score et états du jeu, programmation orientée objet appliquée. Côté IA : agent à règles, algorithme minimax pour le morpion, Q-learning sur FrozenLake, entraînement PPO sur CartPole.

```
import pygame

pygame.init()
ecran = pygame.display.set_mode((800, 600))
horloge = pygame.time.Clock()
joueur = pygame.Rect(380, 520, 40, 40)

en_cours = True
while en_cours:
```

```
    for evt in pygame.event.get():
        if evt.type == pygame.QUIT:
            en_cours = False
    touches = pygame.key.get_pressed()
    joueur.x += (touches[pygame.K_RIGHT] - touches[pygame.K_LEFT]) * 6
    ecran.fill((20, 33, 61))
    pygame.draw.rect(ecran, (245, 184, 46), joueur)
    pygame.display.flip()
    horloge.tick(60)
pygame.quit()
```

TP7 — Jeu 2D avec agent IA. Partie 1 : programmer un jeu au choix (Snake, Pong ou casse-briques) avec score et écran de fin. Partie 2 : ajouter un adversaire contrôlé par l'ordinateur, à règles ou par minimax. Partie 3 : entraîner un agent PPO sur CartPole avec Stable-Baselines3 et afficher sa partie.

Séance 8 — Soutenances (2h)

Chaque équipe présente son projet : 10 minutes de démonstration en direct, puis 5 minutes de questions. Avec 6 à 7 équipes, la séance tient en 1h45 ; les 15 dernières minutes servent au bilan du module et au retour des étudiants.

Projet, évaluation et ressources

Projet (équipes de 3), lancé en séance 4. Chaque équipe choisit une piste et combine au moins trois bibliothèques vues en cours.

Piste	Exemple de sujet	Bibliothèques attendues
Analyse de données	Tableau de bord sur les prix des denrées au Sénégal	Pandas, Plotly, Streamlit
IA	Classification d'avis clients ou de maladies des plantes à partir de photos	scikit-learn ou PyTorch, Hugging Face, Gradio
Jeux	Jeu 2D avec adversaire intelligent	Pygame, Gymnasium, Stable-Baselines3

Livrables. Dépôt Git avec README, code commenté, environnement reproductible (`requirements.txt` ou `pyproject.toml`) et démonstration fonctionnelle.

Modalités d'évaluation.

Composante	Poids	Détail
Projet	50 %	Code et fonctionnalités 30 %, soutenance 10 %, documentation 10 %
TP rendus	30 %	3 TP notés : TP3 (données), TP5 (IA), TP7 (jeux)
Contrôle écrit (1h, hors volume horaire)	20 %	Lecture de code et questions sur les bibliothèques

Ressources.

- Jake VanderPlas, *Python Data Science Handbook*, disponible gratuitement en ligne.
- Aurélien Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, O'Reilly.
- Al Sweigart, *Invent Your Own Computer Games with Python*, disponible gratuitement en ligne.
- Cours gratuit Hugging Face sur le NLP (huggingface.co/learn).
- Documentations officielles : numpy.org, pandas.pydata.org, scikit-learn.org, pytorch.org, spacy.io, pygame.org, gymnasium.farama.org, docs.streamlit.io.
- Dépôt du cours : github.com/Techbadji/pythonavance.